

# Guia tecnica del proyecto

## Brazo Robotico ABB IRB 14050

Workspace ROS 2, behavior tree, simulacion Gazebo, control web, gamepad, EGM, Jetson y computadora de vision/voz.

**Repositorio:** Brazo-Robotico

**Workspace ROS:** irb14050\_ws

**Aplicacion web:** web\_control\_hub

**Objetivo:** entender como se comunican todos los componentes para poder operar, depurar y presentar el sistema.

## Indice

---

1. Resumen del sistema
2. Estructura del repositorio
3. Conceptos base: ROS 2, nodos, topicos, mensajes y actions
4. Mapa de conexiones fundamentales
5. Arquitectura completa en laptop con simulacion
6. Arquitectura distribuida: Jetson + computadora con GPU
7. Behavior tree: el cerebro del robot
8. Backend FastAPI y ROS gateway
9. Frontend Next.js
10. Control por gamepad tipo Xbox
11. EGM, MoveIt y robot real
12. Vision, voz y percepcion 3D
13. Comandos para lanzar cada perfil
14. Comandos de diagnostico
15. Errores comunes
16. Checklist de presentacion
17. Resumen final



|                            |                                                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| robot_task_manager         | Contiene el behavior tree, nodos auxiliares, launch files y configuraciones de secuencias.                                  |
| robot_web_interface        | Puente ROS que convierte topicos web como /web/teleop_twist y /web/sequence_id en RobotCommand.                             |
| robot_voice_interface      | Parser de comandos de voz/texto. Publica RobotCommand a partir de frases como "agarra el cubo azul" o "paro de emergencia". |
| robot_xbox_teleop          | Lee sensor_msgs/Joy desde joy_node y publica teleop seguro como XBOX_TELEOP.                                                |
| abb_irb14050_egm           | Comunicacion real con el controlador ABB por EGM, gripper por RWS, ejecutores para MoveIt y nodos ligeros para Jetson.      |
| abb_irb14050_gazebo        | Lanza Gazebo y el robot simulado con controladores.                                                                         |
| abb_irb14050_moveit_config | Configuracion MoveIt: SRDF, kinematics, controladores y planning group.                                                     |
| robot_perception           | YOLO, mascara, point cloud y centroides 3D para detectar objetos.                                                           |
| realsense_d415_brin<br>gup | Launch y parametros de la RealSense D415.                                                                                   |

### 3. Conceptos base de ROS 2 usados en este proyecto

---

#### Nodo

Un nodo es un proceso ROS. Por ejemplo, robot\_task\_tree, web\_command\_bridge, egm\_bridge, gamepad\_command\_bridge y move\_group son nodos.

#### Topico

Un topico es un canal de comunicacion asincrono. Un nodo publica y otro se suscribe. Ejemplo: el backend publica un Twist en /web/teleop\_twist; web\_command\_bridge se suscribe y lo convierte a RobotCommand.

#### Action

Un action es para tareas que tardan y tienen feedback. MoveIt y el nodo MoveJoint usan actions porque mover el brazo puede tardar segundos y puede cancelarse.

## Mensajes centrales

| Mensaje          | Uso                                                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| RobotCommand     | Entrada comun al behavior tree. Indica tipo de comando, prioridad, fuente, joint values, secuencia, twist de teleop, etc.         |
| RobotStatus      | Salida comun del behavior tree. Informa modo actual, progreso, si hay ESTOP, si el brazo esta ocupado, texto de estado y errores. |
| Detection3D      | Representa un objeto detectado con clase, color, confianza y pose 3D.                                                             |
| MoveJoint.action | Action simple para mover a un vector de 7 articulaciones cuando no se quiere usar MoveIt directamente.                            |

## 4. Mapa de conexiones fundamentales

Esta tabla resume las conexiones que hacen que el sistema realmente se comuniquen. Si algo falla, normalmente se depura siguiendo estas rutas.

| Ruta               | Publica                | Topico/API                                                                     | Consume         | Resultado                                                                         |
|--------------------|------------------------|--------------------------------------------------------------------------------|-----------------|-----------------------------------------------------------------------------------|
| Frontend a backend | Next.js                | HTTP<br>http://localhost:8000, WebSocket /ws/*                                 | FastAPI         | El operador puede mandar comandos y recibir estado en tiempo real.                |
| Backend a ROS      | RosGateway             | /web/teleop_twist,<br>/web/sequence_id,<br>/voice/text,<br>/robot_task/command | ROS graph       | La API web entra al mundo ROS 2.                                                  |
| Web bridge a BT    | web_command_bridge     | /robot_task/command                                                            | robot_task_tree | Teleop y secuencias web se convierten a RobotCommand.                             |
| Voz a BT           | voice_command_parser   | /robot_task/command                                                            | robot_task_tree | Frases parseadas se vuelven comandos de movimiento, pick, cancel, ESTOP o RESUME. |
| Gamepad a BT       | gamepad_command_bridge | /robot_task/command y /xbox/deadman                                            | robot_task_tree | Teleop con deadman y botones de seguridad.                                        |
| BT a web           | robot_task_tree        | /robot_task/status                                                             | RosGateway y    | Barra de progreso, modo,                                                          |

|                 |                                        |                                          |                        |                                                  |
|-----------------|----------------------------------------|------------------------------------------|------------------------|--------------------------------------------------|
|                 | ee                                     |                                          | frontend por WebSocket | errores, ESTOP activo y estado del brazo.        |
| BT a Gazebo     | robot_task_tree / MoveIt               | Actions de MoveIt y controladores Gazebo | Gazebo controllers     | El robot simulado se mueve.                      |
| BT a robot real | robot_task_tree / MoveIt / EGM directo | /joint_command                           | egm_bridge             | El controlador ABB recibe setpoints por EGM.     |
| Percepcion a BT | object_cloud_bridge                    | /perception/detections_3d                | robot_task_tree        | Pick/place puede seleccionar objetos detectados. |

## Variables de entorno utiles

| Variable                | Donde se usa                                                | Ejemplo                                           |
|-------------------------|-------------------------------------------------------------|---------------------------------------------------|
| ROS_DOMAIN_ID           | ROS 2 DDS entre laptop, Jetson y GPU PC.                    | export ROS_DOMAIN_ID=14                           |
| NEXT_PUBLIC_BACKEND_URL | Frontend Next.js para saber a que backend llamar.           | NEXT_PUBLIC_BACKEND_URL=http://192.168.125.5:8000 |
| ROS_IMAGE_TOPIC         | Backend FastAPI si se quiere transmitir imagen al frontend. | ROS_IMAGE_TOPIC=/camera/color/image_raw           |
| FRONTEND_ORIGINS        | CORS del backend.                                           | FRONTEND_ORIGINS=http://localhost:3000            |

## 5. Arquitectura completa en laptop con simulacion

En la laptop se corre el sistema completo para desarrollo: Gazebo, MoveIt, behavior tree, web bridge, voz, backend y frontend. Este perfil sirve para probar sin robot fisico.

```
Frontend Next.js (localhost:3000)
|
| HTTP/WebSocket
v
Backend FastAPI (localhost:8000)
|
| ROS publishers/subscribers via rclpy
v
ROS graph
|
| /web/teleop_twist, /web/sequence_id, /voice/text
```

```

v
web_command_bridge / voice_command_parser
|
| /robot_task/command
v
robot_task_tree (behavior tree)
|
| MoveIt actions / gripper trajectory / servo topics
v
Gazebo + MoveIt + controllers

```

## Launch principal para laptop

Archivo: `irb14050_ws/src/robot_task_manager/launch/full_system_gazebo.launch.py`

Este launch inicia:

- Gazebo con el ABB IRB 14050 simulado.
- MoveIt para planificar y ejecutar trayectorias.
- `robot_task_tree` como cerebro.
- `web_command_bridge`.
- `voice_command_parser`.
- Opcionalmente el puente de gamepad.

## 6. Arquitectura distribuida: Jetson + computadora con GPU

Para el robot real, la arquitectura se divide por carga computacional y latencia:

- **Jetson Orin NX 8GB:** behavior tree, EGM, gripper, teleop/gamepad, web bridge y opcionalmente MoveIt headless.
- **Computadora con GPU:** RealSense, YOLO, pointcloud, voz y conversion de detecciones 3D.

| Computadora GPU          | Jetson Orin NX                    |
|--------------------------|-----------------------------------|
| -----                    | -----                             |
| RealSense D415           | robot_task_tree                   |
| yolo_node                | egm_bridge                        |
| pointcloud_node          | egm_moveit_executor o egm_direct  |
| voice_command_parser     | egm_joint_jog_servo               |
| object_cloud_bridge      | gamepad_command_bridge + joy_node |
|                          |                                   |
| ROS_DOMAIN_ID compartido | EGM UDP/RWS                       |
| +----- ROS 2 DDS -----   | +----> ABB controller             |

**Importante:** ambas computadoras deben estar en la misma red y con el mismo ROS\_DOMAIN\_ID. Si no, los topicos ROS 2 no se descubren entre maquinas.

## 7. Behavior tree: el cerebro del robot

El nodo principal es robot\_task\_tree. Esta en:

robot\_task\_manager/robot\_task\_manager/task\_tree\_node.py. Su trabajo no es solo ejecutar, sino tambien decidir que comandos acepta y en que orden.

### Responsabilidades principales

- Suscribirse a /robot\_task/command.
- Suscribirse a /perception/detections\_3d.
- Suscribirse a /web/heartbeat para teleop web.
- Suscribirse a /xbox/deadman para teleop por gamepad.
- Publicar estado en /robot\_task/status.
- Publicar comandos de gripper en /gripper/command.
- Publicar teleop en /servo/twist\_cmd.
- Usar MoveIt, acciones o simulacion segun el backend configurado.

### Fragmento representativo

```
self.command_sub = self.create_subscription(
    RobotCommand,
    "/robot_task/command",
    self._command_callback,
    10,
)

self.status_pub = self.create_publisher(
    RobotStatus,
    "/robot_task/status",
    10,
)
```

### Tipos de comando

| command_type | Origen comun         | Que hace                                       |
|--------------|----------------------|------------------------------------------------|
| WEB_TELEOP   | Frontend/backend/web | Stream de teleoperacion desde la interfaz web. |

bridge

|              |                        |                                                           |
|--------------|------------------------|-----------------------------------------------------------|
| XBOX_TELEOP  | Gamepad                | Teleoperacion segura con deadman.                         |
| MOVE_JOINT   | Web, voz o backend     | Mueve una articulacion o un vector completo de 7 joints.  |
| RUN_SEQUENCE | Frontend de secuencias | Carga y ejecuta una rutina definida en YAML.              |
| PICK_OBJECT  | Voz/percepcion         | Selecciona un objeto detectado y ejecuta pick/place.      |
| ESTOP        | Web, voz o gamepad     | Activa paro de emergencia logico y detiene el movimiento. |
| RESUME       | Web, voz o gamepad     | Libera el estado de paro de emergencia logico.            |
| CANCEL       | Web, voz o gamepad     | Cancela tarea activa y vuelve a estado seguro.            |

## Arbitraje y seguridad

El archivo `command_utils.py` define prioridades. La idea es que comandos criticos como ESTOP y CANCEL puedan preemptar. Teleop tiene prioridad alta, y comandos largos como secuencias o pick/place se bloquean si el brazo esta ocupado.

```
COMMAND_PRIORITY_RANK = {  
    "ESTOP": 1,  
    "CANCEL": 1,  
    "XBOX_TELEOP": 2,  
    "WEB_TELEOP": 3,  
    "MOVE_JOINT": 4,  
    "RUN_SEQUENCE": 5,  
    "PICK_OBJECT": 6,  
}
```

## 8. Backend FastAPI y ROS gateway

El backend vive en `web_control_hub/backend/app`. Es una API HTTP/WebSocket que traduce acciones del frontend a publicaciones ROS. La clase clave es `RosGateway`, que crea un nodo ROS dentro del proceso de FastAPI.

### Endpoints importantes

| Endpoint | Metodo | Uso                                                |
|----------|--------|----------------------------------------------------|
| /health  | GET    | Verifica si el backend esta vivo y muestra topicos |



configurados.

|                                    |               |                                                     |
|------------------------------------|---------------|-----------------------------------------------------|
| /robot/state                       | GET           | Devuelve ultima lectura de /joint_states.           |
| /robot/task-status                 | GET           | Devuelve ultimo /robot_task/status.                 |
| /teleop/enable,<br>/teleop/disable | POST          | Habilita o deshabilita teleoperacion desde backend. |
| /teleop/joint-target               | POST          | Envia objetivo completo de joints como MOVE_JOINT.  |
| /teleop/twist                      | POST          | Publica un Twist en /web/teleop_twist.              |
| /teleop/gripper                    | POST          | Ejecuta secuencia open_gripper o close_gripper.     |
| /sequence/run                      | POST          | Publica el ID de secuencia en /web/sequence_id.     |
| /task/estop                        | POST          | Publica RobotCommand(command_type="ESTOP").         |
| /task/resume                       | POST          | Publica RobotCommand(command_type="RESUME").        |
| /ws/robot-state                    | WebSock<br>et | Stream de estado de joints al frontend.             |
| /ws/task-status                    | WebSock<br>et | Stream de estado del behavior tree al frontend.     |

## Ejemplo: gripper desde el frontend

```
Frontend button
-> POST /teleop/gripper {"command": "open"}
-> backend RosGateway.publish_gripper_sequence("open")
-> publica /web/sequence_id = "open_gripper"
-> web_command_bridge crea RobotCommand RUN_SEQUENCE
-> robot_task_tree carga secuencia open_gripper
-> publish_gripper_command("open")
-> /gripper/command
```

## 9. Frontend Next.js

El frontend vive en web\_control\_hub/src. Es la interfaz que el operador usa para teleoperar, lanzar secuencias, ver estado y enviar paro de emergencia.

## Vistas principales

| Ruta            | Funcion                                                                         |
|-----------------|---------------------------------------------------------------------------------|
| /teleoperation  | Panel para habilitar teleop, mandar objetivos de joints y abrir/cerrar gripper. |
| /sequences      | Lanzar rutinas y ver progreso real usando /ws/task-status.                      |
| /vision-voice   | Interfaz para comandos de voz/texto y vision.                                   |
| AppShell global | Contiene navegacion y boton global Emergency Stop/Resume System.                |

## Boton de emergencia global

El componente app-shell.tsx escucha /ws/task-status. Si estop\_active es verdadero, el boton cambia a Resume System; si no, muestra Emergency Stop.

## 10. Control por gamepad tipo Xbox

El paquete robot\_xbox\_teleop convierte mensajes /joy a comandos del behavior tree. El nodo nuevo es gamepad\_command\_bridge.

## Flujo

```
Control fisico
-> Linux /dev/input/js0
-> joy_node
-> /joy (sensor_msgs/Joy)
-> gamepad_command_bridge
-> /robot_task/command (XBOX_TELEOP)
-> /xbox/deadman (Bool)
-> robot_task_tree
-> /servo/twist_cmd
-> Gazebo servo simulado o egm_joint_jog_servo en Jetson
```

## Concepto de deadman

El deadman es un boton que se debe mantener presionado para que el robot se mueva. Si se suelta, el sistema publica cero y el behavior tree deja de transmitir teleop. En la config Jetson se usan por default LB o RB.

## Config importante

- config/gamepad.yaml: perfil generico para laptop.
- config/gamepad\_jetson.yaml: perfil de Jetson con botones de ESTOP, RESUME y gripper.

## 11. EGM, MoveIt y robot real

EGM es la comunicacion de baja latencia con el controlador ABB. El nodo egm\_bridge habla por UDP con el robot. Publica /joint\_states y recibe /joint\_command.

### Flujo real con MoveIt

```
robot_task_tree
-> MoveIt move_group action
-> FollowJointTrajectory
-> egm_moveit_executor
-> /joint_command
-> egm_bridge
-> UDP EGM
-> ABB controller
```

### Flujo ligero egm\_direct

```
robot_task_tree
-> MoveJoint.action /arm/move_joint
-> egm_move_joint_action_server
-> /joint_command
-> egm_bridge
-> ABB controller
```

### Teleop en Jetson

```
gamepad_command_bridge
-> XBOX_TELEOP
-> robot_task_tree
-> /servo/twist_cmd
-> egm_joint_jog_servo
-> /joint_command
-> egm_bridge
```

**Seguridad:** EGM envia comandos reales al robot. Antes de probar, confirmar area despejada, velocidad baja, deadman funcionando, ESTOP fisico disponible y RAPID/EGM configurado hacia la IP correcta de

la Jetson.

## 12. Vision, voz y percepcion 3D

---

La percepcion esta pensada para correr en la computadora con GPU. La Jetson recibe resultados ya procesados por ROS 2.

### Vision

```
RealSense D415
-> /camera/camera/color/image_raw
-> yolo_node
-> /perception/detections
-> pointcloud_node + /camera/camera/depth/color/points
-> /perception/object_clouds
-> object_cloud_bridge
-> /perception/detections_3d
-> robot_task_tree
```

### Voz

```
Texto de voz o input manual
-> /voice/text
-> voice_command_parser
-> /robot_task/command
-> robot_task_tree
```

## 13. Comandos para lanzar cada perfil

---

### Preparacion comun

```
cd ~/Brazo-Robotico/irb14050_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

### Laptop: simulacion completa con Gazebo

```
cd ~/Brazo-Robotico/irb14050_ws
source /opt/ros/jazzy/setup.bash
```

```
source install/setup.bash
```

```
ros2 launch robot_task_manager full_system_gazebo.launch.py \  
  launch_rviz:=false \  
  render_engine:=ogre
```

## Laptop con control conectado

```
ros2 launch robot_task_manager full_system_gazebo.launch.py \  
  launch_rviz:=false \  
  render_engine:=ogre \  
  launch_gamepad_joy:=true \  
  joy_dev:=/dev/input/js0
```

## Backend web

```
cd ~/Brazo-Robotico/web_control_hub  
source ~/Brazo-Robotico/irb14050_ws/install/setup.bash  
  
python3 -m uvicorn backend.app.main:app --host 0.0.0.0 --port 8000
```

## Frontend apuntando a backend en otra maquina

```
cd ~/Brazo-Robotico/web_control_hub  
  
# Ejemplo: backend corriendo en la Jetson o en otra laptop  
export NEXT_PUBLIC_BACKEND_URL=http://192.168.125.5:8000  
  
PATH="$HOME/.local/node/current/bin:$PATH" npm run dev
```

## Frontend web

```
cd ~/Brazo-Robotico/web_control_hub  
PATH="$HOME/.local/node/current/bin:$PATH" npm run dev  
  
# Abrir en navegador:  
# http://localhost:3000
```

## Sistema mock sin Gazebo

```
ros2 launch robot_task_manager full_system_mock.launch.py
```

## Jetson: stack real ligero recomendado

```
cd ~/Brazo-Robotico/irb14050_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export ROS_DOMAIN_ID=14

ros2 launch robot_task_manager full_system_jetson.launch.py
```

## Jetson con IP del controlador ABB distinta

```
ros2 launch robot_task_manager full_system_jetson.launch.py \
  controller_ip:=192.168.125.1 \
  egm_rx_port:=6511 \
  egm_tx_port:=6510
```

## Jetson en modo aun mas ligero: sin MoveIt

```
ros2 launch robot_task_manager full_system_jetson.launch.py \
  motion_backend:=egm_direct \
  launch_moveit:=false \
  launch_egm_moveit_executor:=false \
  launch_egm_direct_action:=true
```

Este modo sirve para teleop y movimientos por joints. Para pick/place cartesiano con planeacion se recomienda usar `motion_backend:=abb_moveit`.

## Computadora con GPU: vision y voz remotas

```
cd ~/Brazo-Robotico/irb14050_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export ROS_DOMAIN_ID=14

ros2 launch robot_task_manager remote_ai_workstation.launch.py
```

## Solo gamepad standalone

```
ros2 launch robot_xbox_teleop gamepad_teleop.launch.py
```

## Ver mapeo del control

```
ros2 topic echo /joy
```

## 14. Comandos de diagnostico

| Comando                                          | Para que sirve                                      |
|--------------------------------------------------|-----------------------------------------------------|
| <code>ros2 node list</code>                      | Ver nodos activos.                                  |
| <code>ros2 topic list -t</code>                  | Ver topicos y tipos de mensaje.                     |
| <code>ros2 topic echo /robot_task/status</code>  | Ver estado del behavior tree.                       |
| <code>ros2 topic echo /robot_task/command</code> | Ver comandos que entran al behavior tree.           |
| <code>ros2 topic echo /joint_states</code>       | Ver estado actual de joints.                        |
| <code>ros2 topic echo /xbox/deadman</code>       | Confirmar si el deadman del control esta activo.    |
| <code>ros2 topic hz /joint_states</code>         | Medir frecuencia de feedback del robot/simulador.   |
| <code>ros2 action list</code>                    | Ver actions disponibles.                            |
| <code>curl http://localhost:8000/health</code>   | Ver si el backend esta conectado y que topicos usa. |

## Publicar comandos manuales para pruebas

```
# Paro de emergencia
ros2 topic pub --once /robot_task/command robot_task_msgs/msg/RobotCommand \
"{source: manual, command_type: ESTOP, priority: 100.0}"

# Resume
ros2 topic pub --once /robot_task/command robot_task_msgs/msg/RobotCommand \
"{source: manual, command_type: RESUME, priority: 100.0}"

# Abrir gripper por topico bajo nivel
```

```
ros2 topic pub --once /gripper/command std_msgs/msg/String "{data: open}"
```

## 15. Errores comunes y como pensarlos

| Sintoma                              | Posible causa                                                           | Que revisar                                                                        |
|--------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| El frontend abre pero no mueve nada. | Backend no esta corriendo o no esta sourced con ROS.                    | <code>curl /health</code> , logs de Uvicorn, <code>ros2 topic list</code> .        |
| El BT no recibe comandos web.        | Falta <code>web_command_bridge</code> o backend publica en otro topico. | <code>ros2 topic echo /web/sequence_id</code> y <code>/robot_task/command</code> . |
| El gamepad no mueve.                 | No llega <code>/joy</code> o no se presiona <code>deadman</code> .      | <code>ros2 topic echo /joy</code> y <code>/xbox/deadman</code> .                   |
| La Jetson no ve los nodos de la GPU. | Dominio ROS distinto, red bloqueada o multicast DDS limitado.           | <code>ROS_DOMAIN_ID</code> , misma red, firewall, <code>ros2 node list</code> .    |
| EGM no conecta.                      | RAPID no esta corriendo o <code>UC_DEVICE</code> apunta a otra IP.      | IP de Jetson, puerto 6511, logs de <code>egm_bridge</code> .                       |
| Movelt espera action server.         | No esta <code>egm_moveit_executor</code> o el controlador no coincide.  | <code>ros2 action list</code> y <code>launch</code> de Jetson/ABB.                 |

## 16. Checklist de presentacion

1. Explicar que el behavior tree es el cerebro y punto unico de seguridad.
2. Mostrar que web, voz y gamepad se convierten a RobotCommand.
3. Mostrar que el BT publica RobotStatus para el frontend.
4. Explicar la diferencia entre simulacion en laptop y ejecucion real en Jetson.
5. Explicar por que vision/voz van en la computadora con GPU.
6. Explicar EGM: feedback por `/joint_states`, comandos por `/joint_command`, UDP hacia ABB.
7. Demostrar ESTOP y RESUME desde el frontend.
8. Demostrar deadman del gamepad con `/xbox/deadman`.
9. Mostrar una secuencia y la barra de progreso leyendo `/robot_task/status`.



## 17. Resumen final

---

Este proyecto esta organizado para que la logica critica viva en ROS 2 y no en la interfaz. El frontend es la cara visual; el backend es el puente HTTP/WebSocket a ROS; el behavior tree es el cerebro; MoveIt/Gazebo o EGM son los backends de movimiento; vision y voz son fuentes de comandos o percepcion; el gamepad es teleop de baja latencia con deadman.

La separacion laptop/Jetson/GPU permite desarrollar con simulacion completa en una computadora, y despues desplegar el lazo critico en la Jetson para el robot real sin cargarla con vision ni voz.